InkLab-Vela

FreeRTOS → Zephyr RTOS

Complete Porting Guide | Target: STM32G0B1 (v1 PCB)

Written for v1 hardware — designed so the H562 v2 lands on ready code

0. Document Overview & How to Use This Guide

This guide is the complete porting plan for InkLab-Vela: migrating the firmware from FreeRTOS (on the STM32G0B1 v1 PCB, which is the active development board right now) to Zephyr RTOS. Every step in this guide targets the G0B1 v1 PCB. The design decisions are intentionally made so that when the STM32H562 v2 PCB arrives, the codebase is already 80%+ complete — the H562 migration will be a board DTS swap and a Kconfig adjustment, not a rewrite.

Primary target: STM32G0B1 (v1 PCB) — Secondary target: STM32H562 (v2 PCB)

All code in this guide compiles and runs on the v1 board with the G0B1.

Every module is written to be HAL-free in its public API so it recompiles for the H562 with zero logic changes.

Hardware-specific differences (clock speeds, DMA channels, TrustZone) are isolated to the board DTS overlay and prj.conf — not in module .c files.

The v2 migration checklist is in Section 3 Phase E.

How to navigate

Sections 1-2: Environment setup and GitHub/repository strategy. Do these before writing any firmware.

Section 3: Module porting order — peripheral by peripheral, with explicit pass criteria.

Section 4: Protocol migration (JSON → COBS binary).

Section 5: Modularity rules and documentation standards.

Section 6: The master AI session prompt — copy-paste into any new conversation.

1. Tools & Prerequisites

1.1 Host Machine Requirements

Tool	Purpose / Install Command
Python 3.10+	Required by west and Zephyr scripts
CMake 3.20+	Build system — brew install cmake / apt install cmake
Ninja	Fast build backend — brew install ninja / apt install ninja-build
west (Zephyr meta-tool)	pip3 install west
Zephyr ARM toolchain	Download from docs.zephyrproject.org/latest/develop/toolchains/
git 2.30+	Standard version control
VS Code	Recommended IDE
nRF Connect for VS Code	VS Code extension — provides build/flash/debug GUI
J-Link or STLink probe	For v1 flashing; BMP FPGA core replaces this on v2

Tool	Purpose / Install Command
OpenOCD 0.12+	Debug server — only needed if not using nRF Connect

1.2 VS Code Extensions

- nRF Connect for VS Code (Nordic Semiconductor) — primary build/debug extension
- C/C++ IntelliSense (Microsoft)
- Cortex-Debug — for raw GDB sessions if needed
- GitLens — enhances branch and blame workflows
- Error Lens — inline compiler errors

> **Important: Abandon STM32CubeIDE for all Zephyr work**
> CubeIDE and CubeMX are incompatible with the Zephyr build model. All pin configuration, peripheral setup,
> and clock trees are described in Device Tree (.dts) files — not in GUI tools.
> Keep CubeMX only as a reference schematic viewer. Never generate code from it for the Zephyr branch.

2. Repository Strategy — InkLab-Vela

2.1 Repository Context

The Zephyr port lives in a new, dedicated repository: InkLab-Vela. The original InkLab FreeRTOS project is preserved inside it as Legacy/ — a read-only reference folder that is never compiled or linked. It exists so you can open original HAL .c files side-by-side with new Zephyr modules during porting.

> **Why a separate repo (not a branch)?**
> The Zephyr workspace model (west init) restructures the directory at a level that conflicts with CubeMX projects.
> A clean repo prevents accidental HAL code from being pulled into the Zephyr build via CMakeLists.txt.
> git log stays clean — no FreeRTOS noise mixed with Zephyr port history.

2.2 Branch Setup

```
# One-time repo setup
git clone https://github.com/your-org/InkLab-Vela
cd InkLab-Vela
```

```
# main stays buildable at all times — no direct commits
# per-module short-lived branches:
git checkout -b port/power-monitor
# ... port, test, pass criterion ...
git checkout main
git merge --no-ff port/power-monitor -m "port(power_monitor): replace HAL I2C with
Zephyr i2c_burst_read_dt"
git branch -d port/power-monitor

# Long-running risky branches stay isolated:
git checkout -b feat/wamr-integration
git checkout -b feat/binary-protocol
```

2.2 Recommended Branch Naming

Branch	Purpose
main	Always-buildable integration — v1 G0B1 target — primary development branch
port/<module>	Short-lived per module (power-monitor, fpga-spi, usb-cdc, fatfs, led, joystick)
feat/wamr-integration	Long-running — WAMR task — merged only after heap audit passes
feat/binary-protocol	Long-running — COBS telemetry — merged only after full protocol parity
v2/h562-bringup	Created when v2 PCB arrives — board DTS + Kconfig changes only

2.3 Commit Message Convention

```
port(power_monitor): replace HAL_I2C with Zephyr i2c_burst_read_dt
port(fpga): add SPI1 device node to inklab_v1.overlay
port(usb-cdc): migrate CDC_Transmit_FS to uart_fifo_fill
docs(bq25798): add @brief @param @retval Doxygen to all public functions
fix(telemetry): correct EMA coefficient after binary protocol switch
chore(west): update Zephyr SDK to 0.17.0
feat(wamr): initial WAMR task scaffold
v2(dts): add inklab_v2.overlay for STM32H562 pin assignments
```

2.4 GitHub Actions CI

Add CI early. The workflow builds against the G0B1 Nucleo board (closest Zephyr-supported target) and additionally smoke-tests the H562 config so v2 DTS errors are caught on main before the board arrives.

```
# .github/workflows/zephyr-build.yml
name: Zephyr Build — InkLab-Vela
```

```
on: [push, pull_request]
jobs:
  build-v1:
    runs-on: ubuntu-latest
    container: ghcr.io/zephyrproject-rtos/ci:latest
    steps:
      - uses: actions/checkout@v4
      - run: west init -l src
      - run: west update
      - run: west build -b nucleo_g0b1re src -- -DCONF_FILE=prj.conf
      - run: west build -b nucleo_h562zi src -- -DCONF_FILE=prj_h562.conf
```

2.5 Actual Repository Structure — InkLab-Vela

This is the real folder layout as it exists. The west workspace root is one level above the repo root. west init -l src is run from inside the InkLab-Vela/ directory, pointing west at the src/ subdirectory where west.yml lives.

```
InkLab-Vela/                          # Repo root — also where west init is run
  Legacy/                             # Read-only FreeRTOS reference — NOT compiled
    .settings/
    Core/                             # Original CubeMX Core/ with all HAL .c/.h
    Drivers/
    FATFS/
    Middlewares/
    USB_Device/
    inklab-dashboard/                 # Original Vue.js DevConsole
    rtl/                              # Original iCE40 Verilog
    .cproject  .mxproject  .project   # CubeMX project files — reference only
    FPGAPinsCapacitance.csv
    InkLab_V3.0_(port).ioc            # CubeMX config — open for pin reference only
    PCB_updates.txt  Pinout.txt
    STM32G0B1CEUX_FLASH.ld            # Original linker scripts — reference only
    STM32G0B1CEUX_RAM.ld

  boards/                             # Zephyr board overlay files
    inklab_v1.overlay                 # G0B1 v1 — I2C, SPI, USB, SD, GPIOs
    # inklab_v2.overlay               # Added when H562 v2 PCB arrives

  build/                              # Generated by west build — GITIGNORED
    CMakeFiles/  Kconfig  app/
    modules/  zephyr/
    .ninja_deps  .ninja_log
    .vscode-nrf-connect.json          # Also generated — do not commit
    CMakeCache.txt  build.ninja
    cmake_install.cmake  compile_commands.json
    ram.json  rom.json
    sysbuild_modules.txt  zephyr_modules.txt  zephyr_settings.txt

  docs/                               # Project documentation
    InkLab_Project_Reference_v2...    # Hardware reference PDF
    InkLab_Zephyr_Porting_Guide...    # This document
```

```
  src/                                # Zephyr application source — YOUR CODE
    CMakeLists.txt                    # Lists all .c files for the Zephyr build
    prj.conf                          # Kconfig — enable/disable subsystems
    west.yml                          # Declares Zephyr version (pinned tag)
    llm_conv.py                       # AI session context helper script
    summary_LEGACY_MCU.txt            # Auto-summary of Legacy/ HAL code
    summary_UI.txt                    # Auto-summary of dashboard composables
    summary_ZEPHYR.txt                # Auto-summary of ported Zephyr modules
```

Legacy/ is a reference, not a dependency
Legacy/ is never added to CMakeLists.txt. It is never included in any #include path.
Its only purpose is to let you open the original HAL .c files side-by-side with the new Zephyr port in VS Code. Treat it like a read-only PDF of the original code.

build/ and west-managed dirs are always gitignored
Add to .gitignore: build/ .west/ zephyr/ modules/
The build/ directory alone can exceed 500 MB. Never commit it.
The compile_commands.json inside build/ is regenerated on every west build.
VS Code IntelliSense picks it up automatically — no manual configuration needed.

src/ sub-directory growth strategy
src/ starts flat. Add sub-directories only when a second file of the same type arrives.
Suggested split points: src/hal/ when bq25798.c + powerMonitor.c both exist;
src/proto/ when cobs.c + inklab_proto.c both exist; src/app/ for logic modules.
Do not pre-create empty directories — an empty directory communicates nothing.

2.6 west.yml — Locking the Zephyr Version

The west.yml in src/ pins the exact Zephyr revision. Never use a floating branch — always pin a release tag. The entire team builds against identical Zephyr source this way.

```
# src/west.yml
manifest:
  projects:
    - name: zephyr
      url: https://github.com/zephyrproject-rtos/zephyr
      revision: v3.7.0          # Pin to a release tag — never use "main"
      import: true              # Pulls in all Zephyr HAL shims and modules
  self:
    path: InkLab-Vela
```

3. Module Porting Order — Step by Step

Every phase targets the v1 PCB (STM32G0B1). The G0B1 is Cortex-M0+ with no FPU, no TrustZone, and 144 KB RAM — the most constrained the codebase will ever be. Writing clean, HAL-free, DTS-driven code on the G0B1 means the H562 migration (Phase E) is purely mechanical: swap the overlay, adjust Kconfig, done.

> **v1 G0B1 → v2 H562: what actually changes at migration time**
> Board overlay: inklab_v1.overlay → inklab_v2.overlay (new pin assignments, clock tree)
> prj.conf: add CONFIG_ARM_TRUSTZONE_M=y, CONFIG_FPU=y, CONFIG_DUAL_BANK_FLASH=y
> RAM budget: 144 KB → 640 KB — WAMR integration unlocks at this point
> DMA channels: verify no conflicts on H562 GPDMA (different than G0B1 DMA1)
> USB: G0B1 USB FS shared → H562 dedicated USB FS peripheral (DTS node changes)
> Everything else: zero changes if the modularity rules in Section 5 were followed

Phase A — Toolchain & Board Bring-Up (No module code yet)

A.1 — West workspace init & blinky on nucleo_g0b1re

Goal: confirm the Zephyr toolchain can build and flash to the G0B1. Use the Nucleo-G0B1RE eval board as the proxy target until the inklab_v1.overlay is complete. Do not start writing module code until this passes.

```
# Run from inside InkLab-Vela/
west init -l src
west update
west build -b nucleo_g0b1re zephyr/samples/basic/blinky
west flash
```

- Pass criterion: LED blinks at 1 Hz on the Nucleo. If it does not, fix toolchain before anything else.

A.2 — Author inklab_v1.overlay

Write the Device Tree overlay for the v1 PCB. Open Legacy/InkLab_V3.0_(port).ioc in CubeMX as a read-only pin reference. Map every assigned pin to its DTS equivalent. This file is the single source of truth for all hardware assignments in the Zephyr build.

```
// boards/inklab_v1.overlay
// Target: STM32G0B1CEUT6 on InkLab v1 PCB
// Reference: Legacy/InkLab_V3.0_(port).ioc and Legacy/Pinout.txt

/ {
    chosen {
        zephyr,console = &usart1;
```

```
        zephyr,shell-uart = &usart1;
    };
    aliases {
        led0 = &rgb_red;
    };
    leds {
        compatible = "gpio-leds";
        rgb_red: led_r { gpios = <&gpioa 5 GPIO_ACTIVE_LOW>; }; /* adjust pin */
    };
};

&i2c1 {
    status = "okay";
    clock-frequency = <I2C_BITRATE_FAST>;  /* 400 kHz */
    pinctrl-0 = <&i2c1_scl_pb6 &i2c1_sda_pb7>;
    pinctrl-names = "default";
    power_monitor_0: ina219@40 {
        compatible = "ti,ina219";
        reg = <0x40>;
        shunt-resistor = <100000>; /* 100 mOhm in micro-Ohms */
    };
};

&spi1 {
    status = "okay";
    pinctrl-0 = <&spi1_sck_pa5 &spi1_miso_pa6 &spi1_mosi_pa7>;
    pinctrl-names = "default";
    cs-gpios = <&gpioa 4 GPIO_ACTIVE_LOW>;
    fpga_spi: fpga@0 {
        compatible = "vnd,spi-device";
        reg = <0>;
        spi-max-frequency = <8000000>;
    };
};
```

- Pass criterion: west build -b nucleo_g0b1re src compiles without DTS errors. No "missing alias" warnings.

A.3 — GPIO output test (LED on actual v1 PCB)

Before touching any peripheral driver, verify gpio_pin_set_dt() works on the real v1 board. This confirms the overlay pin assignment, the GPIO clock enable, and the flash/debug toolchain are all correct.

```
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

static const struct gpio_dt_spec led =
    GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);

int main(void) {
    gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
    while (1) {
        gpio_pin_toggle_dt(&led);
```

```
        k_sleep(K_MSEC(500));
    }
}
```

- Pass criterion: LED toggles at 1 Hz. Oscilloscope confirms timing.

Phase B — Core Peripherals

B.1 — I2C: port powerMonitor.c (INA219)

This is the reference module for the entire port. It is pure I2C + RTOS mutex — no DMA, no interrupts, no complex state. Port it first to establish the HAL-to-Zephyr translation pattern that all subsequent drivers will follow.

STM32 HAL (before)	Zephyr API (after)
HAL_I2C_Mem_Read(&hi2c1, addr, reg, 1, buf, len, HAL_MAX_DELAY)	i2c_burst_read_dt(&i2c_dev, reg, buf, len)
HAL_I2C_Mem_Write(&hi2c1, addr, reg, 1, buf, len, HAL_MAX_DELAY)	i2c_burst_write_dt(&i2c_dev, reg, buf, len)
osMutexAcquire(i2c1_mutex, osWaitForever)	k_mutex_lock(&i2c1_mutex, K_FOREVER)
osMutexRelease(i2c1_mutex)	k_mutex_unlock(&i2c1_mutex)
osDelay(10)	k_sleep(K_MSEC(10))

The PowerMonitor_t handle struct stores a const struct device * (from DEVICE_DT_GET) instead of I2C_HandleTypeDef *. Initialize it in the module init function, not in main.

- Pass criterion: Power readings appear over RTT/UART within 2% of known good values.

B.2 — I2C: port bq25798.c

Same pattern as powerMonitor.c but more registers. Port the I2C read/write calls first. Do not touch BQ25798_GetJson() yet — that gets replaced in Phase D.

- Pass criterion: BQ25798_ReadAll() returns valid voltage/current/status over USB shell.

B.3 — SPI + DMA: port fpga.c

The FPGA communicates over SPI with DMA. This is the most performance-sensitive peripheral on the platform. Get the SPI working without DMA first, then add DMA.

```
/* Step 1 — polled SPI, no DMA */
struct spi_buf tx_buf = { .buf = pkt, .len = len };
struct spi_buf_set tx_set = { .buffers = &tx_buf, .count = 1 };
spi_transceive_dt(&fpga_spi, &tx_set, NULL);
```

```
/* Step 2 — async SPI with DMA */
/* Enable CONFIG_SPI_ASYNC=y in prj.conf */
spi_transceive_signal_dt(&fpga_spi, &tx_set, NULL, &done_signal);
k_poll(&done_event, 1, K_FOREVER);
```

Performance note: SPI DMA path

The FreeRTOS version uses HAL_SPI_Transmit_DMA with a binary semaphore in the DMA complete callback.

In Zephyr, use spi_transceive_signal_dt + k_poll on a k_poll_signal. This is equivalent throughput

with zero HAL overhead. Confirm DMA channel assignment does not conflict with SD card SPI.

- Pass criterion: FPGA_Program_Slot(0) loads bitstream correctly. Logic analyzer confirms SPI framing.

B.4 — GPIO inputs: Joystick

The 5-way joystick is polled via ADC or GPIO depending on v1 wiring. Port the read function and confirm DTS alias resolves.

```
/* If analog: use Zephyr ADC subsystem */
#include <zephyr/drivers/adc.h>
static const struct adc_dt_spec joy_adc =
    ADC_DT_SPEC_GET_BY_NAME(DT_PATH(joystick), ud);

/* If GPIO matrix: */
static const struct gpio_dt_spec joy_up =
    GPIO_DT_SPEC_GET(DT_NODELABEL(joy_up), gpios);
```

- Pass criterion: All 5 directions register in the DevConsole Device IO tab.

B.5 — PWM: LED Manager

The RGB LED uses PWM. Replace TIM HAL calls with the Zephyr PWM API.

```
#include <zephyr/drivers/pwm.h>
static const struct pwm_dt_spec led_r =
    PWM_DT_SPEC_GET(DT_ALIAS(pwm_led_r));

pwm_set_dt(&led_r, PWM_MSEC(1), PWM_USEC(brightness_r));
```

- Pass criterion: LED Manager patterns (idle, busy, error) are visually correct.

Phase C — RTOS Task Architecture

C.1 — Thread definitions and priorities

In Zephyr, threads are defined with K_THREAD_DEFINE() at link time or k_thread_create() at runtime. Replace all osThreadNew() calls. Keep the same priority hierarchy — do not optimize priorities until the full system is running.

FreeRTOS (before)	Zephyr (after)
osThreadNew(USB_Task, NULL, &usb_attr)	K_THREAD_DEFINE(usb_thread, 2048, usb_task_fn, NULL, NULL, NULL, 5, 0, 0)
osThreadNew(FPGA_Task, NULL, &fpga_attr)	K_THREAD_DEFINE(fpga_thread, 2048, fpga_task_fn, NULL, NULL, NULL, 4, 0, 0)
osThreadNew(Telemetry_Task, ...)	K_THREAD_DEFINE(telem_thread, 1024, telem_task_fn, NULL, NULL, NULL, 6, 0, 0)
osThreadNew(Joystick_Task, ...)	K_THREAD_DEFINE(joy_thread, 512, joy_task_fn, NULL, NULL, NULL, 7, 0, 0)
osThreadNew(LED_Task, ...)	K_THREAD_DEFINE(led_thread, 512, led_task_fn, NULL, NULL, NULL, 8, 0, 0)

C.2 — Message queues and mutexes

```
/* FreeRTOS */
osMessageQueueId_t telem_q = osMessageQueueNew(8, sizeof(TelemMsg_t), NULL);
osMessageQueuePut(telem_q, &msg, 0, osWaitForever);
osMessageQueueGet(telem_q, &msg, NULL, osWaitForever);

/* Zephyr equivalent */
K_MSGQ_DEFINE(telem_q, sizeof(TelemMsg_t), 8, 4);
k_msgq_put(&telem_q, &msg, K_FOREVER);
k_msgq_get(&telem_q, &msg, K_FOREVER);

/* Mutex */
K_MUTEX_DEFINE(i2c1_mutex);
k_mutex_lock(&i2c1_mutex, K_FOREVER);
k_mutex_unlock(&i2c1_mutex);
```

C.3 — USB CDC: port USB Task

This is the most impactful single change for the researcher-facing interface. The FreeRTOS version calls CDC_Transmit_FS() from usbd_cdc_if.c (CubeMX generated). In Zephyr, USB CDC ACM is a UART device node.

```
/* prj.conf */
CONFIG_USB_DEVICE_STACK=y
CONFIG_USB_CDC_ACM=y
CONFIG_UART_INTERRUPT_DRIVEN=y

/* In source */
#include <zephyr/usb/usb_device.h>
```

```
#include <zephyr/drivers/uart.h>

static const struct device *cdc_dev =
    DEVICE_DT_GET(DT_NODELABEL(cdc_acm_uart0));

/* Transmit (replaces CDC_Transmit_FS) */
uart_fifo_fill(cdc_dev, (uint8_t*)buf, len);

/* Receive callback (replaces USB_ReceiveCallback in usbd_cdc_if.c) */
uart_irq_callback_user_data_set(cdc_dev, cdc_rx_cb, NULL);
uart_irq_rx_enable(cdc_dev);
```

8KB RX ring buffer

The existing 8KB static ring buffer in app_freertos.c can be kept exactly as-is.
Only the fill function changes from CDC_Receive_FS callback to uart_irq_callback.
The ring buffer and command parser (frontend_api.c) are zero-change.

- Pass criterion: Type any command in DevConsole Terminal. Device responds. No character drops under continuous telemetry stream.

C.4 — SD card / FATFS

Zephyr wraps ELM FatFs under its own disk/FS subsystem. The f_open / f_read / f_write calls at the application level are identical — only the mount and init sequence changes.

```
/* prj.conf */
CONFIG_FAT_FILESYSTEM_ELM=y
CONFIG_DISK_DRIVER_SDMMC=y
CONFIG_FILE_SYSTEM=y

/* Replaces app_fatfs.c MX_FATFS_Init() */
#include <zephyr/fs/fs.h>

static struct fs_mount_t sd_mount = {
    .type = FS_FATFS,
    .fs_data = NULL,
    .mnt_point = "/SD:",
};
fs_mount(&sd_mount);

/* All f_open / f_read / f_write calls below this point: UNCHANGED */
```

The custom user_diskio_spi.c (low-level SPI SD driver) can be reused as a Zephyr disk driver shim or replaced by CONFIG_DISK_DRIVER_SDMMC if the SD is wired to an SDMMC peripheral. On v1 (SPI-wired SD), keep user_diskio_spi.c and register it as a Zephyr disk backend.

- Pass criterion: FPGA slot scan finds all bitstream files on first boot. Write + read speed test passes.

Phase D — Validation & Performance Gates

D.1 — Heap & Stack audit (critical before WAMR)

Run this before committing to WAMR integration. Zephyr provides CONFIG_THREAD_ANALYZER to dump per-thread stack usage.

```
/* prj.conf */
CONFIG_THREAD_ANALYZER=y
CONFIG_THREAD_ANALYZER_USE_PRINTK=y
CONFIG_THREAD_ANALYZER_AUTO=y
CONFIG_THREAD_ANALYZER_AUTO_INTERVAL=10

/* Output (example) */
  usb_thread      : unused 412 / total 2048  (20% used)
  fpga_thread     : unused 680 / total 2048  (33% used)
  telem_thread    : unused 512 / total 1024  (50% used)
```

Target: each thread should have at least 30% stack headroom at peak load. Reduce oversized stacks before adding WAMR.

- Gate: Total committed RAM (threads + queues + static) must be under 490 KB, leaving 150 KB for WAMR interpreter.

D.2 — Telemetry throughput test

Verify the USB CDC ACM path can sustain the same throughput as the FreeRTOS version. Use the existing Power Profiler tab as the benchmark tool.

- Pass criterion: Power Profiler streams 4 rails at 10 Hz with no missed frames. Measured with Wireshark USB capture.

D.3 — FPGA bitstream slot switching under load

Slot switching must be tested while the telemetry task is active. This validates the SPI mutex protects against race conditions between the FPGA task and the telemetry task.

- Pass criterion: 100 back-to-back slot switches without a single corrupt bitstream. No FPGA nDONE failures.

D.4 — BMP/SWD core validation

Load the SWD RTL bitstream and verify the FPGA can program a known-good SWD target through the BMP wrapper Zephyr task.

- Pass criterion: 64 KB firmware upload completes without error at >= 100 KB/s throughput.

Phase E — v2 H562 Migration (When the PCB Arrives)

If Phases A-D were completed cleanly on the G0B1, the H562 migration is a half-day task, not a week-long effort. The entire firmware logic stays identical. Only the three items below change.

> **Phase E is mechanical, not creative — if you need to rewrite logic here, a modularity rule was broken**
> Every hour spent fixing abstraction leaks during Phase E is a sign that a HAL header crept into a public .h, or a peripheral address was hardcoded in a .c file.
> The correct Phase E experience: swap overlay, adjust Kconfig, build succeeds, flash, test.

E.1 — Add inklab_v2.overlay

Create boards/inklab_v2.overlay using the v2 schematic. The structure is identical to inklab_v1.overlay — same node names, different pin assignments and peripheral instances (SPI3 instead of SPI1, USB FS dedicated peripheral, I2C2, etc.).

```
// boards/inklab_v2.overlay
// Target: STM32H562RIV on InkLab v2 PCB
// Reference: InkLab_Project_Reference_v2 (docs/) + v2 schematic

&i2c2 {  /* H562 uses I2C2 for sensors — adjust to match schematic */
    status = "okay";
    clock-frequency = <I2C_BITRATE_FAST>;
    power_monitor_0: ina4235@40 { ... };
};

&spi3 {  /* FPGA now on SPI3 */
    status = "okay";
    cs-gpios = <&gpioa 15 GPIO_ACTIVE_LOW>;
    fpga_spi: fpga@0 { ... };
};
```

E.2 — Add prj_h562.conf

Create a second Kconfig file for the H562. It includes prj.conf and adds H562-specific symbols. Do not modify prj.conf — keep it the shared baseline.

```
# src/prj_h562.conf
# H562-specific additions on top of prj.conf

CONFIG_FPU=y                    # Hardware FPU — enables float in hot paths
CONFIG_ARM_TRUSTZONE_M=y        # TrustZone — required for secure boot later
CONFIG_FLASH_PAGE_LAYOUT=y      # Dual-bank OTA support
CONFIG_MCUBOOT_BOOTLOADER_MODE_SWAP_SCRATCH=y  # OTA swap mode

# WAMR unlocks here — heap budget now allows it
CONFIG_WASM_MICRO_RUNTIME=y
CONFIG_WAMR_HEAP_SIZE=153600    # 150 KB for WAMR interpreter
```

```
# H562 USB dedicated peripheral
CONFIG_USB_DEVICE_STACK=y       # Same Kconfig key — Zephyr handles the HW diff
```

E.3 — DMA channel audit

The H562 uses GPDMA (General Purpose DMA) with a different channel model than the G0B1's DMA1. Verify in the v2 overlay that FPGA SPI3 DMA and SD card SPI DMA are assigned to different GPDMA channels. This is the most likely source of a subtle bug on v2.

- Pass criterion: All Phase D validation tests pass on v2 hardware. FPGA slot switching and SD card access work simultaneously without corruption.

Phase	Target MCU	Status
A — Toolchain + GPIO	G0B1 (v1)	Do this now
B — I2C, SPI, PWM, ADC	G0B1 (v1)	Do this now
C — RTOS tasks, USB, FATFS	G0B1 (v1)	Do this now
D — Validation gates	G0B1 (v1)	Do this now
E — Overlay + Kconfig swap	H562 (v2)	When v2 PCB arrives
WAMR integration	H562 (v2) only	After Phase E passes

4. Protocol Migration: JSON → COBS-Framed Binary

4.1 Why JSON Is the Wrong Choice for This System

The current protocol sends newline-delimited JSON for all telemetry. Looking at the Vue frontend (systemStore.js), the processIncomingData() function handles two distinct stream types that are already incompatible:

- Structured telemetry — { "type": "ina", ... } parsed with JSON.parse()
- Unstructured plaintext — slot names, bench results, FW info, SD log lines parsed with startsWith() and regex

This mixed approach has real costs: text framing is 3-5x larger than binary for float data, JSON.parse on every telemetry frame adds browser GC pressure at high rates, partial frame corruption silently drops data (try/catch eats errors), and text regexes are fragile maintenance burdens.

4.2 Recommended Replacement: COBS-Framed Binary Packets

COBS (Consistent Overhead Byte Stuffing) is the correct protocol for this use case. It is used in production embedded telemetry systems (ArduPilot MAVLink, Zephyr MCUMGR, OpenLog Artemis) for exactly these reasons:

Property	COBS Binary vs JSON Text
Frame delimiter	0x00 byte — unambiguous, no escaping needed
Overhead	1 byte per 254 bytes of payload (< 0.4%)
Float encoding	4 bytes exact — JSON uses 8-14 bytes per float
Parse complexity	memcpy into struct — zero allocation, zero GC
Corruption detection	CRC16 field per packet — detectable, not silent
Partial frame handling	Discard on 0x00 boundary — clean recovery
Mixed stream problem	Eliminated — all messages are typed binary packets

4.3 Packet Structure

```
/* inklab_proto.h — shared between firmware and a TS/JS code-gen script */

#pragma pack(push, 1)
typedef struct {
    uint8_t  msg_type;    /* See InkLabMsgType_t enum */
    uint16_t seq;         /* Wrapping sequence number — detect drops */
    uint16_t payload_len; /* Bytes in payload[] */
    uint8_t  payload[];   /* Variable-length, type-specific struct */
    /* uint16_t crc16;    appended after payload by framer */
} InkLabPacket_t;
#pragma pack(pop)

typedef enum {
    MSG_INA_TELEM      = 0x01,
    MSG_BQ_TELEM       = 0x02,
    MSG_SYS_STATUS     = 0x03,
    MSG_GPIO_STATE     = 0x04,
    MSG_FPGA_SLOT      = 0x05,
    MSG_LOG_STRING     = 0x10, /* UTF-8 string — replaces plain text lines */
    MSG_CMD_RESPONSE   = 0x11, /* ACK/NAK with optional payload */
    MSG_BENCH_RESULT   = 0x12,
} InkLabMsgType_t;
```

4.4 Migration Strategy — No Big Bang

Do not rewrite everything at once. The frontend already dispatches on data.type for JSON and on startsWith() for plaintext. Add a mode flag:

1. Add CONFIG_PROTO_BINARY=n to prj.conf. Default off — JSON still works.
2. Implement the COBS framer as a standalone module (cobs.c / cobs.h). Zero dependencies.
3. Add a USB command PROTO:SET:BINARY to enable binary mode at runtime.
4. In the Vue frontend, detect binary frames by checking if the first byte of a received chunk is 0x00 or a known type byte (not ASCII {). Route to a binary parser.

5. Port telemetry types one by one: INA first (most frequent), then BQ, then SYS_STATUS.
6. Keep MSG_LOG_STRING for all existing plaintext lines — this eliminates the startsWith() fragmentation.
7. Once all types are binary, set CONFIG_PROTO_BINARY=y as default and remove JSON formatters.

4.5 Frontend Integration

The processIncomingData() function in systemStore.js currently does: if line starts with { → JSON.parse. Replace with:

```
// In useSerial.js — raw byte handler
function processRawBytes(chunk: Uint8Array) {
  cobsDecoder.push(chunk);  // cobsDecoder buffers until 0x00 delimiter
}

// cobsDecoder emits complete decoded packets:
cobsDecoder.on("packet", (buf: Uint8Array) => {
  const msgType = buf[0];
  const seq     = (buf[2] << 8) | buf[1];   // little-endian
  const paylen  = (buf[4] << 8) | buf[3];
  const payload = buf.slice(5, 5 + paylen);
  const crc     = (buf[5 + paylen + 1] << 8) | buf[5 + paylen];

  if (!verifyCrc16(buf.slice(0, 5 + paylen), crc)) return; // drop corrupt

  binaryHandlers[msgType]?.(payload, store);
});
```

Backward compatibility

During migration, the firmware can emit both formats: JSON when PROTO:SET:JSON is active,
binary when PROTO:SET:BINARY. The frontend auto-detects based on the first byte of each WebSerial
chunk (0x7B = ASCII { means JSON, anything else means binary). No breaking change.

5. Modularity Rules & Documentation Standards

5.1 The Three Laws of InkLab Modules

Law 1 — No HAL in the header

Public .h files must not include stm32g0xx_hal.h, stm32h5xx_hal.h, or any RTOS header. Headers expose types and function signatures only. HAL types (I2C_HandleTypeDef, etc.) are
replaced by opaque const struct device * in the Zephyr port. This makes every module

compilable in a host-side unit test without a cross-compiler.

Law 2 — One module, one responsibility
A module init function initializes exactly one hardware resource or logical subsystem. powerMonitor.c initializes INA sensors. bq25798.c initializes the charger. Neither touches the other's peripheral. Shared I2C bus access is mediated only by a mutex defined in a dedicated bus_manager.c — never by one module calling into another's internals.

Law 3 — Config, not magic numbers
Every constant that a researcher might want to change (telemetry rate, PGA gain, bitstream slot
count, WAMR heap size) is a #define or Kconfig symbol in a single config header.
No raw numbers in .c files. No "I'll just hardcode 16 here" comments.

5.2 Directory Layout (Enforced)

```
app/
  src/
    hal/          # Hardware abstraction — one file per IC (bq25798.c, ina4235.c)
    drivers/      # Peripheral drivers — fpga.c, sd_card.c, epd.c
    rtos/         # Task definitions — usb_task.c, fpga_task.c, telem_task.c
    proto/        # Protocol layer — cobs.c, inklab_proto.c, frontend_api.c
    app/          # Application logic — battery_soc.c, led_manager.c, sys_power.c
    wasm/         # WAMR host — wasm_host.c, native_bindings.c
  include/
    hal/          # Public HAL headers
    drivers/
    rtos/
    proto/
    app/
    wasm/
  boards/
    inklab_v1.dts
    inklab_v2.dts
  prj.conf
  CMakeLists.txt
```

5.3 Mandatory Documentation Standard

Every public function (declared in a .h file) must have a Doxygen comment block. No exceptions. Private (static) functions in .c files should have at minimum a one-line description.

```
/**
 * @brief  Initialize the INA power monitor for a specific rail.
```

```
 *
 * Configures the INA device, calibrates the current LSB based on the
 * provided shunt resistor value, and verifies the device is reachable
 * on the I2C bus. Must be called before PowerMonitor_Read().
 *
 * @param  dev         Pointer to the PowerMonitor handle to initialize.
 * @param  i2c_dev     Zephyr I2C device pointer (from DEVICE_DT_GET).
 * @param  addr        7-bit I2C address of the INA device.
 * @param  r_shunt_ohms  Physical shunt resistor value in Ohms.
 * @param  max_expected_A  Maximum expected current in Amperes (sets PGA range).
 *
 * @retval 0           Success.
 * @retval -ENODEV     Device not found on I2C bus.
 * @retval -EIO        I2C communication error during calibration write.
 *
 * @note   This function is NOT thread-safe. Call from a single init context
 *         before RTOS threads start, or protect with a mutex.
 */
int PowerMonitor_Init(PowerMonitor_t *dev, const struct device *i2c_dev,
                      uint8_t addr, float r_shunt_ohms, float max_expected_A);
```

5.4 Return Value Convention

All functions that can fail must return int (0 = success, negative errno = failure). Never return HAL_StatusTypeDef in the Zephyr port. Use standard errno values: -EIO for bus errors, -EINVAL for bad parameters, -ENOMEM for allocation failures. This makes error codes portable and compatible with Zephyr logging macros.

5.5 Zephyr-Specific Rules

- Never call k_sleep() or k_msleep() from an interrupt context. Use k_work or k_timer instead.
- Never use __disable_irq() / __enable_irq() directly. Use Zephyr IRQ lock API: irq_lock() / irq_unlock().
- Mark all thread stack arrays with K_THREAD_STACK_DEFINE, not plain arrays. This ensures correct alignment on Cortex-M33 with TrustZone.
- All I2C and SPI device handles must be acquired at module init time with DEVICE_DT_GET, not inside task loops. Fail fast at boot, not at runtime.
- GPIO toggle safety floor remains 5 ms (100 Hz max). Document this explicitly in frontend_api.h with a @note Doxygen comment on the GPIO command handler.

6. Master AI Session Prompt Template

Copy the following block into any new AI conversation before asking a firmware question. Fill in the [CURRENT TASK] section. The more specific you are about what you are attempting right now, the more precise the AI guidance will be.

```
=== INKLAB-VELA PROJECT CONTEXT ===

REPOSITORY: InkLab-Vela (github.com/your-org/InkLab-Vela)

HARDWARE — ACTIVE DEV TARGET (v1 PCB):
  - MCU: STM32G0B1CEUT6, Cortex-M0+, 64 MHz, 256 KB Flash, 144 KB RAM
  - FPGA: Lattice iCE40UP5K — connected via SPI1 with DMA
  - Power: BQ25798 (USB-C 5V) + INA219 x4 current monitors
  - Storage: SD card via SPI + FATFS — 16 named FPGA bitstream slots
  - Display: E-paper (EPD) via parallel 8-bit from FPGA

HARDWARE — NEXT TARGET (v2 PCB — not yet arrived):
  - MCU: STM32H562RIV, Cortex-M33, 250 MHz, 2 MB Flash (dual-bank), 640 KB RAM
  - Migration: board overlay swap + Kconfig adjustments only — no logic rewrites
  - Unlocks: WAMR (needs 150 KB RAM headroom), TrustZone, HW FPU, ETM trace

RTOS STATUS:
  - Currently running on v1: FreeRTOS (CMSIS-RTOS2) — source in Legacy/
  - Porting to: Zephyr RTOS — source in src/
  - Active tasks: USB, FPGA, Telemetry, Joystick, LED
  - Planned tasks (post-v2): EPD, Wasm (WAMR)

REAL FOLDER STRUCTURE:
  InkLab-Vela/
    Legacy/         — Original FreeRTOS CubeMX project — READ ONLY, never compiled
      Core/         — HAL .c/.h files (reference for porting)
      Drivers/  FATFS/  Middlewares/  USB_Device/
      inklab-dashboard/ — Original Vue.js DevConsole
      rtl/          — iCE40 Verilog source
    boards/
      inklab_v1.overlay — G0B1 pin assignments (I2C, SPI, USB, SD, GPIO)
    build/          — Generated by west build — GITIGNORED
    docs/           — This guide and the hardware reference PDF
    src/            — Zephyr application source (YOUR CODE)
      CMakeLists.txt
      prj.conf      — Kconfig
      west.yml      — Pinned Zephyr version
      llm_conv.py   — AI session context helper
      summary_LEGACY_MCU.txt  summary_UI.txt  summary_ZEPHYR.txt

USB INTERFACE:
  - Full-speed CDC — currently JSON newline framing
  - Migrating to COBS-framed binary (inklab_proto.h — planned in src/)
  - 8 KB RX ring buffer — command parser in frontend_api.c

WEB FRONTEND:
  - Vue.js DevConsole in Legacy/inklab-dashboard/ — WebSerial API
  - DO NOT suggest rewriting the frontend — it is not in scope

MODULARITY RULES (ENFORCE THESE IN ALL GENERATED CODE):
  - No HAL headers (#include "stm32g0xx_hal.h" etc.) in public .h files
  - All public functions must have @brief @param @retval @note Doxygen comments
  - Return int with errno values — never HAL_StatusTypeDef
```

```
  - Config values in #define or Kconfig — never raw numbers in .c files
  - One module = one responsibility — no cross-module direct calls

KEY SOURCE FILES (in Legacy/Core/ — use as porting reference):
  app_freertos.c   — task init (becomes K_THREAD_DEFINE in src/)
  frontend_api.c   — USB command parser (zero-change in port)
  powerMonitor.c   — INA219 driver (THIS is the reference port module)
  bq25798.c        — charger driver
  fpga.c           — SPI/DMA FPGA interface
  pinmux.c         — replaced by boards/inklab_v1.overlay

KNOWN RISKS:
  1. G0B1 has 144 KB RAM — no WAMR on v1, budget is tight even without it
  2. G0B1 is M0+ with no FPU — avoid float-heavy code in hot paths
  3. SPI shared between FPGA and SD card — verify DMA channel assignments in
overlay
  4. .noinit section: check Zephyr linker script puts it outside BSS zero-init
  5. GPIO toggle hard floor: 5 ms (100 Hz max) — document in frontend_api.h

=== CURRENT TASK ===
[Describe exactly what you are working on right now.]
[Example: "I am porting powerMonitor.c from Legacy/Core/Src/powerMonitor.c to
 src/power_monitor.c for Zephyr on the G0B1. Replace all HAL_I2C calls with
 Zephyr i2c_burst_read_dt. Keep the public API identical. Add complete Doxygen
 comments to all public functions. Return int errno codes, not
HAL_StatusTypeDef."]

CONSTRAINTS FOR THIS TASK:
  - Target MCU: STM32G0B1 (Cortex-M0+, 144 KB RAM, no FPU)
  - Target RTOS: Zephyr
  - Board overlay: boards/inklab_v1.overlay
  - Files being modified: [list files]
  - Do not change: [list anything that must stay the same]
```

6.1 Prompt Usage Tips

- Always paste the full context block above, not just the CURRENT TASK section. The model needs the full picture to avoid generic advice.
- When porting a module, paste the ENTIRE existing .h and .c file into the prompt after the context block. Do not summarize it.
- If the model suggests a function that you cannot find in the Zephyr docs, ask: "Show me the Kconfig symbol that enables this API and the DTS property it depends on."
- After any AI-generated code, add: "Now add complete Doxygen comments (@brief, @param, @retval, @note) to every public function in this file."
- When debugging: paste the exact west build error output. Do not paraphrase it.

7. Quick Reference: HAL → Zephyr API Cheat Sheet

STM32 HAL / FreeRTOS	Zephyr Replacement
HAL_GPIO_WritePin(port, pin, val)	gpio_pin_set_dt(&spec, val)
HAL_GPIO_ReadPin(port, pin)	gpio_pin_get_dt(&spec)
HAL_I2C_Mem_Read(...)	i2c_burst_read_dt(&dev, reg, buf, len)
HAL_I2C_Mem_Write(...)	i2c_burst_write_dt(&dev, reg, buf, len)
HAL_SPI_Transmit_DMA(...)	spi_transceive_dt(&dev, &tx_set, &rx_set)
HAL_TIM_PWM_Start(...)	pwm_set_dt(&spec, period, pulse)
HAL_ADC_Start / HAL_ADC_PollForConversion	adc_sequence_init_dt + adc_read
osDelay(ms)	k_sleep(K_MSEC(ms))
osThreadNew(fn, arg, &attr)	K_THREAD_DEFINE(id, stack, fn, ...)
osMessageQueueNew(n, sz, attr)	K_MSGQ_DEFINE(q, sz, n, align)
osMessageQueuePut(q, &msg, 0, timeout)	k_msgq_put(&q, &msg, K_MSEC(timeout))
osMessageQueueGet(q, &msg, 0, timeout)	k_msgq_get(&q, &msg, K_MSEC(timeout))
osMutexNew(attr)	K_MUTEX_DEFINE(m) or k_mutex_init(&m)
osMutexAcquire(m, timeout)	k_mutex_lock(&m, K_MSEC(timeout))
osMutexRelease(m)	k_mutex_unlock(&m)
CDC_Transmit_FS(buf, len)	uart_fifo_fill(cdc_dev, buf, len)
f_mount / f_open / f_read (FATFS)	fs_mount + fs_open + fs_read (same ELM FatFs)
__disable_irq() / __enable_irq()	irq_lock() / irq_unlock()
vTaskGetRunTimeStats()	CONFIG_THREAD_ANALYZER=y + thread_analyzer_print()

InkLab Zephyr Porting Guide — End of Document